

Assignment Title

SQL Injection Simulation in Java

Due: 11:59 PM on 09/08/2025

Objectives

- Understand how SQL injection attacks work conceptually.
- Simulate Tautology database attack
- Learn why direct string concatenation in database queries is dangerous.
- Explore the use of parameterized queries or prepared statements as a defense.

Background

- SQL injection (SQLi) is a common security vulnerability where attackers insert malicious input into an SQL query to gain unauthorized access or extract information. This assignment uses a simulated “mock database” (no real DB) to safely demonstrate SQL injection and its mitigation in Java. You are required to simulate Tautology database attack in this project.

Assignment Requirements

1. Create Java Program(s)

- Write a single Java class or a project with multiple classes (your choice).
- Implement a mock database with a list of users (username, password).
- Provide **two login methods**:
 - **Vulnerable login:** Constructs SQL-like queries using string concatenation.
 - **Secure login:** Uses parameterized-style checks (no concatenation).
- In both methods, simulate query execution and print results to the console.

2. Demonstrate SQL Injection

- Show at least one malicious input (e.g., admin' OR '1'='1) that bypasses the vulnerable method but fails in the secure method.
- Output should clearly show:

- The query string executed.
- Whether login succeeded or failed.

3. Optional Bonus (up to 20%)

- Accept user input via Scanner for interactive testing.(10 points)
- Besides tautology, add more than one type of SQL injection attack (e.g., comment injection --).(10 points)

Please submit:

- **Source code (.java)** file(s).
- **Lab report:** Provide a short write-up (1–2 paragraphs) explaining:
 - Word document or pdf document.
 - What SQL injection is.
 - Why the vulnerable method is insecure.
 - How the secure method prevents attacks.
 - Screenshots: Console output of tests.
 - If your running results meet all requirements, please state “my project meets the requirements”. Otherwise please point out the limitations in your project.
- **Please submit all individual files. Do not compress all as a zip file.**

Grading Rubric (100 points)

Criterion	Points
Program correctness	
- Vulnerable login implemented and runs without errors	20
- Secure login implemented and runs without errors	20
- SQL injection attempt works on vulnerable but fails on secure method	20
Code quality	
- Clear variable names, good structure, necessary comments	15

Criterion	Points
Lab report	
- Explanation of SQLi and mitigation	15
- Console output clearly shows query strings and results	10
Bonus (optional)	
- Extra attacks, interactivity, or enhancements	Up to 20

Notes:

- **Do not connect to a real database.** Use only mock objects or in-memory lists.
- **No sensitive data.** This is purely a simulation for learning.

I have provided *PartialCodeProject_1* as a reference. You may complete the project by building on it. Alternatively, you are welcome to work on the project independently without using my code.



PartialCodeProject_
1.txt

When you run your project, the running results may look like this:

```
[VULNERABLE] Executing: SELECT * FROM users WHERE username='alice' AND password='password1'
Normal login (vulnerable): true
[VULNERABLE] Executing: SELECT * FROM users WHERE username='admin' OR '1'='1' AND password='anything'
Injection attempt (vulnerable): true

[SAFE] Checking with parameters
Normal login (safe): true
[SAFE] Checking with parameters
Injection attempt (safe): false
```